Analyzing Surrogate Satellite Workload Performance on Simulated Computer Architectures

BROOKLYN BECK, Virginia Tech, USA

This research compares two potential computing architectures for typical satellite workloads. Several metrics, including time, power use, cost, required support for radiation hardening, and reliability, are used to carefully analyze the impacts of each of the considered architectures. The work simulates the architectures of the Jetson Orin Nano and the Raspberry Pi 5 on gem5 using Virginia Tech's multi-machine systems rlogin.cs.vt.edu. Three workloads are tested on these simulated architectures: a matrix multiplication program used in class assignments as a benchmark to architectures studied in class, a task schedule repair software written by the student representing real time operation management, and an OpenMP-based multithreaded workload to simulate multiprocessor workloads. Based on the results of this research, the Jetson Orin Nano is the recommended architecture for further use in the student's lab for satellite planning and scheduling autonomy research. The research found that for the specific workloads tested there are benefits for either architecture but that the Raspberry Pi 5 ultimately is the more suitable architecture based on the tested parameters. Further research would compare the simulated architectures to physical hardware.

CCS Concepts: • **Computer systems organization** → Parallel architectures.

Additional Key Words and Phrases: Onboard Computing, Multi-core Processors, Satellite Autonomy, Architecture Simulation

ACM Reference Format:
Brooklyn Beck. 2026. Analyzing Surrogate Satellite Workload Performance on Simulated Computer Architectures. 1, 1 (May 2026), 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The number of satellites in orbit increases exponentially every year, with 4510 individual objects launched into space in 2025 [11]. The computing requirements of each satellite has increased in recent years, with more processing done onboard to optimize the limited communication bandwidth with ground control, transmitting only significant and pre-processed data. Some satellite missions propose including machine learning tools to identify image features. For example, software onboard EO-1 was used to identify cloud cover, lava flow, and wildfires [3]. The onboard processors could react to the image identification, throwing out unusable data such as heavy cloud cover, and taking further images of anomalous events such as volcanic activity, all without direct control from operators.

Beyond data processing, satellites have an increased need for task management autonomy, where less computing bandwidth can be allocated for task sequences. Satellites with autonomous guidance navigation and control (GNC), power management, and fault detection have an advantage compared to satellites without. This autonomy is necessary for satellites beyond Earth orbit that have significant lag between communication times with ground control, but can be applied to Low Earth Orbit (LEO) satellites as well.

Author's Contact Information: Brooklyn Beck, brooklynbeck@vt.edu, Virginia Tech, Blacksburg, Virginia, USA.

ACM XXXX-XXXX/2026/5-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Historically, satellite OBCs have lagged behind their grounded computer counterparts by roughly ten years in terms of performance. Satellite systems have relatively limited access to power and memory, and require radiation hardened components to handle singe-event upsets (SEUs). Additional hardware to handle these SEUs and testing requirements leads to this performance lag between satellite OBCs and standard hardware. Modern satellite missions need to carefully select onboard computers (OBCs) to accommodate both increased autonomy and increased computing expectations. Satellites are expensive to build, launch, and operate. Poor performing OBCs limit the data collected and lower the overall usability of the satellite. Thus, it is especially significant to select a computing architecture that is well suited to the standard workloads handled by the specific mission.

One tool satellite mission planners can employ is the ability to simulate potential computing architectures as part of the OBC selection process. One example of software that allows various architectures to be tested in simulation is gem5. gem5 supports homogeneous and heterogeneous multi-core systems, power and energy modeling, and event-driven memory systems [1], allowing various architectures to be realistically modeled.

This project simulates two potential satellite computer architectures, NVIDIA's Jetson Orin Nano and Raspberry Pi 5, and analyses the performances of the two architectures when running comparable satellite operation tasks. This study steps into the shoes of a satellite system engineer, having to select the optimal computing hardware based on several conflicting metrics and benchmarks. While the results of this study will not influence the hardware of a physical satellite mission, it directly affects the student's PhD research, as based on the results she will recommend one of the two architectures to her advisor for further satellite planning and scheduling autonomy research.

This research is not highly novel, however novelty is not crucial to its success. Other studies have validated gem5 simulations [2], used gem5 to compare architectures for specific workloads [13], modeled Raspberry Pi architectures in gem5 [4], and analyses of specifically low power computing capabilities [10]. Prior studies have shown the importance of multicore OBCs for satellite operations [9] Much can be learned and applied from these earlier studies to improve the results of this specific project.

The rest of the project is outlined as followed: Section 2 gives an overview of the prior literature related to the project topic, Section 3 describes the methodology and general research design, Section 4 describes and analyzes the results of the study, and Section 5 draws conclusions from the work done and recommends further research.

2 Literature Review

The limited computing resources onboard satellites are used for a number of tasks, including: image analysis, anomaly detection, health management, navigation, and ground communication. When computing resources are selected for a satellite, the designers must be aware of hardware, model, and security limitations. Hardware must have low power consumption, reliability against radiation caused errors, a large working memory, and must be capable of working in extreme temperatures [5]. Small satellites often rely on commercial processors because of their availability and low set up cost, but this limits the ability to configure the computer architecture to the needs of the particular mission. Onboard data processing models are limited by a lack of datasets to train on. To address this, many satellites with onboard data processing use a continuous training method, which can be slower and less reliable. Security risks arise when a satellite processes sensitive data, in which case the data must be encrypted and authenticated when transmitted. Power management algorithms, used to address the need for low power consumption, can be model based, requiring WCET, deadline, priority and precedence information, or model free, which are more general purpose. The CPU frequency can be chosen based on a mixture of data from the current system and inferences from a model. Setting the CPU to a lower frequency does not necessarily lower the total energy. When a CPU is idle, the power consumption can be reduced by lowering the voltage, however this reduction can be offset by a large start up power intake. Heat generation is also a factor affected by CPU frequency and voltage draw.

One paper [5] recommended several hardware solutions to the above identified problems, including NVIDIA's Jetson Orin Nano [12]. The Orin Nano has undergone radiation testing, proving that, with aluminum shielding, the hardware had sufficient radiation tolerance for up to 2 years in Low Earth Orbit [14]. Research papers have tested the Orin Nano classifying wildfires from satellite imagery [15]. The Orin Nano is small, ideal for small CubeSat designs. The Orin Nano allows for direct control over task assignment to cores and core power usage, which can be used to optimize satellite operations for low power use, decreased heat generation, and real-time task management based on the computing resources required at the given moment. Additionally, using off the shelf hardware allows us to benchmark our results on standard tests. The Jetson Orin Nano, diagrammed in Figure 1, has a 512-core GPU, a 6-core 64-bit CPU, 4 GB of memory, and an input of 5-10 Watts [8]. Mostly this study focuses on the CPU, but the Orin Nano allows access to peripheral devices and plugins through ethernet as well as having a connected GPU for highly parallelized workloads.

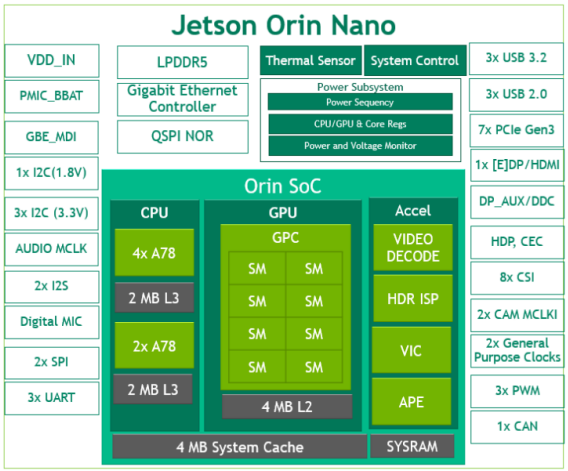

Fig. 1. Block diagram of Jetson Orin Nano [8]

Another potential architecture to address the needs of satellite onboard processing is the Raspberry Pi 5. Raspberry Pi boards have been used in a number of satellite operations, as cataloged in NASA JPL's Guideline Document regarding Raspberry Pi usage in space [7]. These operations include sensor monitoring, remote operations, usage as a desktop PC, file storage, offline image analysis, stop motion or time lapse camera, and to monitor a system. These operations address most if not all priorly discussed requirements for onboard hardware. Most Raspberry Pi boards used in space have been launched into LEO, with lower radiation concerns. Minimal testing has shown that the Raspberry Pi boards are likely to operate for 5 years in LEO. The Raspberry Pi 5 in particular has a 2.4GHz quad-core, 64-bit Arm Cortex-A76 CPU, with 512KB L2 caches and a 2MB shared L3 cache, and a VideoCore VII GPU 2.

3 Methodology

For this research, the main features of the NVIDIA Jetson Orin Nano and the Raspberry Pi 5 are modeled in gem5. Then, the simulated architectures are compared using three workloads simulating satellite operations and proceses, while using Virginia Tech's CPU and multi-machine system rlogin.cs.vt.edu. By analyzing the statistics generated in the simulations, conclusions are drawn comparing the two architectures and their capability of handling satellite operations. All code used in this project is uploaded to GitHub at the address

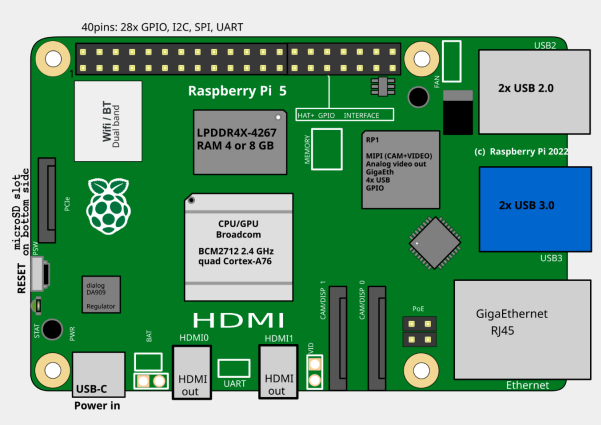

Fig. 2. Block diagram of Raspberry Pi 5 [6]

https://github.com/brooklynbeck/cs5504_finalproject_brooklynbeck which can be accessed to reproduce the results. This GitHub includes compiling instructions, the files for the GERRY workload, the simulation shell script, and stat files from each simulation.

In gem5, several factors can be modeled including the type of CPU model, the ISA, the number of cores, and the memory system [2]. These simulate the hardware types and capabilities that create pipelining and multi-staged instructions depending on the modeled architecture. Table 1 lists the settings determined to match the physical systems. These were determined based on how closely they aligned with the spec sheets for the Raspberry Pi 5 and the Jetson Orin Nano. As the full protocols, systems, and capabilities are not publically available knowledge, the simulated architectures will be approximations and not exact matches. The major differences between the two architectures are the protocols for multicore processing, cache sizes, frequencies, and memory systems.

Table 1. Architecture settings.

	Raspberry Pi 5	Jetson Orin Nano
CPU Type	X86O3CPU	X86O3CPU
CPU Count	4	6
Protocol	MESI_Two_Level	CHI
L1 Cache (D and I)	32KiB	64KiB
L2 Cache	512KiB	2MiB
L3 Cache	4MiB	4MiB
Frequency	2.4GHz	1.5 GHz
Memory	DDR4_2400_8x8	LPDDR5_6400_1x16_BG_BL16

Three workloads are tested on the two models:

(1) Dense matrix multiplication, as provided in class files for prior homework assignments with the code densemv.c. This provides a baseline to compare the simulated architectures against classwork to determine that the simulations are running correctly. This also represents a standard benchmark used in many simulations, using a significant amount of computing resources to multiply a large matrix.
(2) Independent_Writes, an OpenMP-based multithreaded workload used in Homework Assignment 9 with the code independent_writes.c. This workload directly uses the multi-core capabilities of the architectures.

In this code, each of the four threads make independent computations with separate data before serially summing the thread results. Notably, this is the only multithreaded workload and therefore the only one that can take advantage of the multicore architectures.

(3) GERRY, an iterative schedule repair algorithm that mimics real-time onboard satellite planning and scheduling capabilities. The software takes an initial schedule, modifies it using a Monte Carlo simulation to introduce conflicts, and repairs the schedule while maintaining planning constraints. The code stores and iteratively updates long scheduling datasets, making the running hardware swap between blocks of data often.

In the project proposal, another workload was suggested in place of Independent_Writes: YOLO image feature identification. YOLO would have represented onboard image analysis capabilities that are becoming more common in modern satellites. Two factors lead to switching this workload with Independent_Writes. First, a multithreaded workload needed to be included to show the full capabilities of the Raspberry Pi 5 and the Jetson Orin Nano, both of which are multi threaded. In testing, and as will be explained fully in the results section, it became clear that additional cores are just added complexity if the workload does not directly call for multiple threads. The second reason YOLO was removed from the workload list is that the student reached the memory limit allotted to her in the rlogin.cs.vt.edu virtual CPU. On attempting to install YOLO, errors were thrown and the software had to be deleted for the other tests to run. YOLO, while a relatively light software compared to other image processing models, with its dependencies it does require several GB of memory. Thus, it was decided to replace YOLO image feature detection.

The gem5 simulations, running through the two architectures and the three workloads, is controlled by the shell function run_sat_tests.sh, which is a modified version of the simulation script from Homework 9 with updated architecture settings, workloads, and stat file directory locations. Benchmarks of performance are stored in stat files after each simulation, and include simulation time, memory use, and the strengths and weaknesses of each architecture noted in prior literature.

This study has a few limitations. First, this study will be focused exclusively on two architectures, ignoring potentially superior architectures by the scope of the research. Second, there will be no comparison of the results simulated in gem5 to physical hardware, limiting the validity and reproducibility of the results. The settings for each architecture were carefully decided, but cannot fully replicate the physical hardware. Third, the workloads roughly simulate potential satellite operations but are not directly the same programs that a satellite in orbit would run.

4 Results

The Raspberry Pi 5 (abbreviated to rPi5) and Jetson Orin Nano (abbreviated to Jetson) simulated architectures are benchmarked on the following performance markers, as generated by the gem5 simulation per architecture per workload: simulated runtime, instructions per cycle (IPC), cache demand misses, ruby control messages, memory bandwidth, ROB full events, and mispredictions. These statistics lead to further analysis, theorizing why, based on the chosen parameters, one architecture performed better than the other.

On every workload, rPi5 completed the program in fewer simulated seconds, with a speedup from the Jetson of over 1.76. The specific statistics are shown in Table 2. The dense matrix multiplication workload was the largest of the three, with the Jetson taking nearly twice as long to run the program compared to the rPi5. Given the faster clockspeed of the rPi5, this makes intuitive sense. However, there are more features than clockspeed that influence an architecture.

The IPC is a strong marker of CPU performance, calculated as the number of instructions completed every cycle. While the two compared architectures have different clock cycle speeds, the rPi5 at 2.4 GHz and the Jetson at only 1.5 GHz, the instructions per cycle only considers how efficient each individual cycle is. Table 3 shows

Table 2. Simulated Seconds and Speedup of the Raspberry Pi 5 over the Jetson Orin Nano.

	Raspberry Pi 5	Jetson Orin Nano	Speedup
densemv	0.026335	0.048949	1.858705145
independent_writes	0.000320	0.000565	1.765625
gerry	0.000397	0.000710	1.788413098

the calculated IPC for each workload. Note that the workload independent_writes uses four CPU cores, so the IPC shown is an average over the four used cores. Over all three workloads, the rPi5 had a higher IPC, indicating that it was more efficient with each instruction.

Table 3. IPC over the workloads.

	Raspberry Pi 5	Jetson Orin Nano
densemv	0.691418	0.594999
independent_writes	0.7910335	0.72520525
gerry	0.802683	0.717105

Figure 3 separates the independent_writes workload into the four CPUs. CPU0 has the lowest workload of the cores, with the other three CPUs roughly even. On every CPU with this workload, the rPi5 had a higher IPC than the Jetson. Jetson has a total of 6 CPUs, but only used 4 in the simulation.

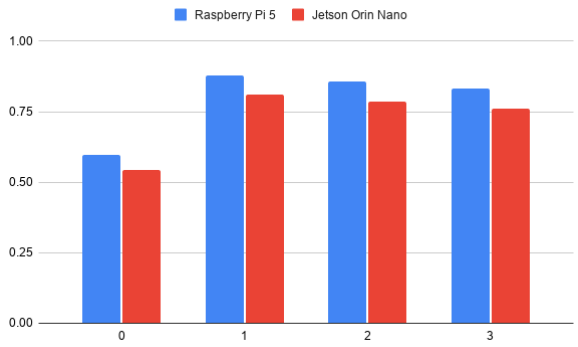

Fig. 3. Enter Caption

One result that in hindsight should have been obvious from the beginning, is that the dense and gerry workloads did not use the additional available computing cores. The workloads only called for a single thread, and thus were executed serially. While this is hardly a novel finding, it is an important reminder that multiple cores is only added complexity if the program does not use them. When running a single program, it is on the programmer to parallelize the program such that it can take advantage of all cores.

Looking specifically at the GERRY workload, the Jetson had fewer mispredictions, with 5044 on the gerry workload compared to 6577 on the rPi5 architecture. Additionally, the Jetson used about half the memory bandwidth at 394 MB/sec on the gerry workload, where the rPi5 had a bandwidth of 704 MB/sec. The rPi5 was able to move more memory every second, likely in part to the different memory models. Another factor of this

difference could be the difference in cache sizes. The Jetson did not need to move as much memory, lowering the bandwidth, because each cache has greater storage capacity.

One statistic to support this is the number of L1 and L2 cache predictions. On the dense and gerry workloads, the two architectures had largely similar demand misses. However, for independent_writes, the Jetson decreased the number of cache misses from 1574 on the rPi5 to 115, over a 10x improvement. This indicates that the Jetson has good performance sharing memory when using multiple cores, not requiring to slowly move memory between caches as often.

The rPi5 had fewer ruby control messages, which tracked the total number of requests between the cores to maintain cache coherence. Even when only one core is in use, each workload had at minimum 17000 control based messages. For the gerry workload, rPi5 only had 44193 protocol coherence messages, compared to 105723 messages, over twice the amount. My initial hypothesis before running the simulations was that the Jetson, with more memory in each cache, would have fewer cache misses and fewer control messages. This is proven to be untrue. With more memory per core, each core communicates their data status far more, even if the cores are idle.

With both architectures executing on an out-of-order CPU model, it is important to observe the number of reorder buffer (ROB), instruction queue (IQ), load queue (LQ), and store queue (SQ) full events. These statistics indicate when structural hazards happen. ROB full event happen when there is a long latency event, such as a cache miss. The instruction queue full events occur when the functional units are full and the instruction queue fills. Load and store full events indicate that the program is slowed by a limit on the memory resources, such as the data bus. On the gerry workload, the Jetson had fewer ROB full events at 2880 compared to 3617 for rPi5, indicating fewer cache misses, which is supported by the earlier analysis of the gerry workload cache misses. The rPi5 simulated architecture has the advantage for IQ, LQ, and SQ full events, indicating that Jetson filled all functional units causing delays. This is strong evidence for why, despite having more cache available, Jetson had a lower IPC. Additional instructions stalled, slowing the overall program.

5 Conclusion

The analysis leads to the conclusion that for the selected workloads, the Raspberry Pi 5 architecture has better performance than the Jetson Orin Nano. The Raspberry Pi 5 has a higher clockspeed and higher memory bandwidth, leading to faster program execution, higher instructions per second, and fewer structural hazards in the hardware. This is different than the initial expectations of these two architectures. It could have reasonably been assumed that with more cores and larger cache per core, that the Jetson would have better performance. This research concludes the obvious: more cores only improve performance if the cores are used. With two of the selected workloads only using one core, and the multithreaded workload only using four cores no matter how many were available, Jetson's additional cores were insignificant if not a hindrance to the performance. With more cores, there was more communication between the cores about cache coherence, slowing the rate of instructions executed and taking up space on the bus.

The analysis is limited to the capabilities of gem5 to model the selected architectures, meaning that the stats do not exactly reflect the physical hardware. Approximations had to be made, such as selecting from gem5's library of coherence policies and memory models. Additionally, there are factors about the architectures that were not captured by the workloads, such as how the architectures would run with parallel workloads.

Future work needs to be done to correct these limitations, such as comparing the gem5 simulations to the physical hardware to improve the accuracy of the simulated architectures. Another future work could test truly parallel workloads on the architectures, determining if the two additional cores on the Jetson make up for the slower clock speed in the long run. Historically, increasing the number of cores has been the next method to improve performance once clock speed stopped showing vast improvement. These tests did not focus on multithreading capabilities.

6 Reproducibility

Overleaf link: https://www.overleaf.com/read/dvwczcwkjgsv#70e55d
GitHub link: https://github.com/brooklynbeck/cs5504_finalproject_brooklynbeck

References

[1] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, et al. 2011. The gem5 simulator. *ACM SIGARCH computer architecture news* 39, 2 (2011), 1–7.

[2] Anastasiia Butko, Rafael Garibotti, Luciano Ost, and Gilles Sassatelli. 2012. Accuracy evaluation of gem5 simulator system. In *7th International workshop on reconfigurable and communication-centric systems-on-chip (ReCoSoC)*. IEEE, 1–7.

[3] Steve Chien, Rob Sherwood, Daniel Tran, Benjamin Cichy, Gregg Rabideau, Rebecca Castano, Ashley Davis, Dan Mandl, Stuart Frye, Bruce Trout, Seth Shulman, and Darrell Boyer. 2005. Using Autonomy Flight Software to Improve Science Return on Earth Observing One. *Journal of Aerospace Computing, Information, and Communication* 2, 4 (2005), 196–216. arXiv:https://doi.org/10.2514/1.12923 doi:10.2514/1.12923

[4] Iago de Oliveira Silvestre, Antonio Carlos Beck Filho, and Leandro Buss Becker. 2021. Using gem5 Simulator to Support Design Space Exploration Targeting ARM Architecture. In *2021 XI Brazilian Symposium on Computing Systems Engineering (SBESC)*. IEEE, 1–8.

[5] Lorenzo Diana and Pierpaolo Dini. 2024. Review on hardware devices and software techniques enabling neural network inference onboard satellites. *Remote Sensing* 16, 21 (2024), 3957.

[6] Efa. 2026. Raspberry Pi. https://en.wikipedia.org/wiki/Raspberry_Pi#/media/File:RaspberryPi_5B_28-08-2024.svg

[7] Steven M Guertin. 2022. Raspberry pis for space guideline. *NASA: Washington, DC, USA* 7 (2022).

[8] Leela Subramaniam Karumbunathan. 2023. Solving entry-level edge AI challenges with Nvidia Jetson Orin Nano. https://developer.nvidia.com/blog/solving-entry-level-edge-ai-challenges-with-nvidia-jetson-orin-nano/

[9] Fisnik Kraja and Georg Acher. 2011. Using many-core processors to improve the performance of space computing platforms. In *2011 Aerospace Conference*. 1–17. doi:10.1109/AERO.2011.5747445

[10] Lucas Martin Wisniewski, Jean-Michel Bec, Guillaume Boguszewski, and Abdoulaye Gamatié. 2022. Hardware solutions for low-power smart edge computing. *Journal of Low Power Electronics and Applications* 12, 4 (2022), 61.

[11] Edouard Mathieu, Pablo Rosado, and Max Roser. 2022. Space exploration and satellites. https://ourworldindata.org/space-exploration-satellites

[12] NVIDIA. 2026. NVIDIA Jetson Nano. https://developer.nvidia.com/embedded/jetson-nano-developer-kit

[13] Umer Shahid, Ayesha Ahmad, and Shanzay Wasim. 2024. gem5-based evaluation of CVA6 SoC: Insights into the Architectural Design. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 298–300.

[14] Windy S Slater, Nayana P Tiwari, Tyler M Lovelly, and Jesse K Mee. 2020. Total ionizing dose radiation testing of NVIDIA Jetson nano GPUs. In *2020 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–3.

[15] Kathiravan Thangavel, Dario Spiller, Roberto Sabatini, Stefania Amici, Sarathchandrakumar Thottuchirayil Sasidharan, Haytham Fayek, and Pier Marzocca. 2023. Autonomous satellite wildfire detection using hyperspectral imagery and neural networks: A case study on australian wildfire. *Remote Sensing* 15, 3 (2023), 720.

Received 23 March 2026; revised 9 May 2026